points of how sketches are organized and generally work and will apply to every sketch that we might write, not just our previous example. We will get to more on the specifics of functions and statements, including their syntax and parameters, a little later.

To begin with, Arduino C follows a set of rules that govern syntax and structure and tell the interface board what to do. Some of these rules are inherited from C and others have been put into place by the Arduino development team to simplify usage for beginners. C is a top-down structural programming language, meaning that lines of code are executed in order from the top of the program until it reaches the end at the bottom. We will adopt this top-down method in our analysis of the sketch for Project 1 to discuss the major points of the Arduino programming syntax and structure, beginning with our first lines of code.

Using Comments

Looking at our first example, the first three lines contain a small descriptive block of text that we refer to as **comments**. Comments are areas of text that are ignored by the compiler, but provide useful notes and documentation that give us a little extra information about the sketch to make the code easier to understand for us humans. Use them to provide information about the project that the sketch is written for, about what certain variables do, what information a function needs to work properly, or what will happen if some part of the code is changed. Because comments take up no memory space on the Arduino board, they should be used generously throughout your sketch and are one of those good habits to have that should be adopted early on. Other people, including your future self, will be happy and thank you. Here's our example:

```
/* Project 1: RGB Blinky
   Uses a single RGB LED to cycle through three colors.
*/
```

This is an example of **block comments**. Anything in between the characters `/*` and `*/` are completely ignored and do not affect the outcome of the program. Block comments can be stretched across multiple lines, creating blocks of information, but need to be balanced with `/*` at the beginning to tell the compiler to ignore the following text until the compiler reaches the closing characters, `*/`. The Arduino development environment conveniently changes comments to a grey color once they have been properly registered. This lets you know that you have entered comments properly or that you still need to close block comments (because your entire sketch suddenly became grey).

In addition to block comments, a second similar form of comments, called **line comments,** are used throughout the example sketch. Line comments are any line of text that begin with the characters `//` and automatically end with the next line of code. Unlike block comments, line comments do not need closing characters and are often used at the end of a statement to provide a little more information about what that line does. This is the case in our first line comment in the previous example, as follows:

```
// sets digital pins as outputs
```

Both types of comments are also pretty useful for disabling parts of a sketch for testing and debugging. Just "comment out" an offending line to see what happens in your code. It's also helpful to make a change in a part of the code but leave the old code in place using comments just in case you want to go back to it later. Take for example the following line:

```
digitalWrite(10, HIGH);
```

This statement could be disabled or otherwise ignored without deleting the line, by adding line comments in front of the statement as follows:

```
// digitalWrite(10, HIGH);
```